

1. Probar Prompts Suelos

Dentro de FreeCAD, abrimos la consola Python y ejecutamos:

```
import sys
sys.path.insert(0, r"D:\Facultad\PET DAV\Fork\DAV\luigiIntegracionV1\GUIFreeCad")
```

Integer:

```
from InputPrompts.IntegerInputPrompt import IntegerInputPrompt
```

```
Prompt = IntegerInputPrompt("Integer", "Deci un entero")
Prompt.Show()
Prompt.ProcessFinalText("uno dos enviar")
print(Prompt.GetResult())
```

Esperado:

```
PromptResult(Success=True, Value=12, Cancelled=False, Error="")
```

```
>>> import sys
>>> sys.path.insert(0, r"D:\Facultad\PET DAV\Fork\DAV\luigiIntegracionV1\GUIFreeCad")
>>> from InputPrompts.IntegerInputPrompt import IntegerInputPrompt
>>> Prompt = IntegerInputPrompt("Integer", "Deci un entero")
>>> Prompt.Show()
>>> Prompt.ProcessFinalText("uno dos enviar")
PromptResult(Success=True, Value=12, Cancelled=False, Error="")
>>> print(Prompt.GetResult())
PromptResult(Success=True, Value=12, Cancelled=False, Error="")
>>>
```

Float:

```
from InputPrompts.FloatInputPrompt import FloatInputPrompt
```

```
Prompt = FloatInputPrompt("Float", "Deci un decimal")
Prompt.Show()
Prompt.ProcessFinalText("cinco coma dos enviar")
print(Prompt.GetResult())
```

Esperado:

```
Value=5.2
```

```
>>> from InputPrompts.FloatInputPrompt import FloatInputPrompt
>>> Prompt = FloatInputPrompt("Float", "Deci un decimal")
>>> Prompt.Show()
>>> Prompt.ProcessFinalText("cinco coma dos enviar")
PromptResult(Success=True, Value=5.2, Cancelled=False, Error="")
>>> print(Prompt.GetResult())
PromptResult(Success=True, Value=5.2, Cancelled=False, Error="")
>>>
```

String:

```
from InputPrompts.StringInputPrompt import StringInputPrompt
```

```
Prompt = StringInputPrompt("Text", "Deci un texto")
Prompt.Show()
Prompt.ProcessFinalText("linea prueba enviar")
print(Prompt.GetResult())
```

Esperado:

Value='linea prueba'

```
>>> from InputPrompts.StringInputPrompt import StringInputPrompt
>>> Prompt = StringInputPrompt("Text", "Deci un texto")
>>> Prompt.Show()
>>> Prompt.ProcessFinalText("linea prueba enviar")
PromptResult(Success=True, Value='linea prueba', Cancelled=False, Error='')
>>> print(Prompt.GetResult())
PromptResult(Success=True, Value='linea prueba', Cancelled=False, Error='')
>>>
```

2. Probar Selección de Objetos

Crear objetos:

```
import FreeCAD as App
```

```
doc = App.newDocument("DAVPromptTest")
box = doc.addObject("Part::Box", "BoxTest")
cylinder = doc.addObject("Part::Cylinder", "CylinderTest")
sphere = doc.addObject("Part::Sphere", "SphereTest")
doc.recompute()
```

Probar Prompt:

```
from InputPrompts.ObjectSelectionInputPrompt import ObjectSelectionInputPrompt
```

```
Prompt = ObjectSelectionInputPrompt("Object", "Selecciona un objeto")
Prompt.Show()
```

```
Prompt.ProcessFinalText("siguiente")
Prompt.ProcessFinalText("enviar")
print(Prompt.GetResult())
```

Esperado:

```
PromptResult(Success=True, Value='CylinderTest', Cancelled=False, Error='')
```

```

>>> import FreeCAD as App
>>> doc = App.newDocument("DAVPromptTest")
>>> # App.setActiveDocument("DAVPromptTest")
>>> # App.ActiveDocument=App.getDocument("DAVPromptTest")
>>> # Gui.ActiveDocument=Gui.getDocument("DAVPromptTest")
>>> # Gui.runCommand('Std_OrthographicCamera',1)
>>> box = doc.addObject("Part::Box", "BoxTest")
>>> cylinder = doc.addObject("Part::Cylinder", "CylinderTest")
>>> sphere = doc.addObject("Part::Sphere", "SphereTest")
>>> doc.recompute()
3
>>> from InputPrompts.ObjectSelectionInputPrompt import ObjectSelectionInputPrompt
>>> Prompt = ObjectSelectionInputPrompt("Object", "Selecciona un objeto")
List loaded with 3 objects. Ready to iterate.

[Advancing] Object 1 of 3
Successfully focused: 'BoxTest'
>>> Prompt.Show()
>>> Prompt.ProcessFinalText("siguiente")

[Advancing] Object 2 of 3
Successfully focused: 'CylinderTest'
PromptResult(Success=False, Value=None, Cancelled=False, Error='')
>>> Prompt.ProcessFinalText("enviar")
PromptResult(Success=True, Value='CylinderTest', Cancelled=False, Error='')
>>> print(Prompt.GetResult())
PromptResult(Success=True, Value='CylinderTest', Cancelled=False, Error='')
>>>

```

3. Probar ParametrerCollector con entradas Simuladas

Esto prueba prompts + parser + validator, sin voz real:

```
from InputPrompts.ParameterCollector import ParameterCollector
```

```
def CreateLine(x1: float, y1: float, x2: float, y2: float, label: str):
    print("Line:", x1, y1, x2, y2, label)
```

```
Collector = ParameterCollector(DelayMs=0)
```

```
Result = Collector.CollectForFunction(
```

```
    CreateLine,
```

```
    SimulatedFinalTexts=[
```

```
        "uno dos enviar",
```

```
        "dos enviar",
```

```
        "cinco enviar",
```

```
        "dos enviar",
```

```
        "linea prueba enviar",
```

```
    ],
```

```
)
```

```
print(Result)
```

Esperado:

PromptResult(Success=True, Value={'x1': 12.0, 'y1': 2.0, 'x2': 5.0, 'y2': 2.0, 'label': 'linea prueba'}, Cancelled=False, Error="")

```
>>> from InputPrompts.ParameterCollector import ParameterCollector
>>> def CreateLine(x1: float, y1: float, x2: float, y2: float, label: str):
...     print("Line:", x1, y1, x2, y2, label)
...
>>> Collector = ParameterCollector(DelayMs=0)
>>> Result = Collector.CollectForFunction(
...     CreateLine,
...     SimulatedFinalTexts=[
...         "uno dos enviar",
...         "dos enviar",
...         "cinco enviar",
...         "dos enviar",
...         "linea prueba enviar",
...     ],
... )
>>> print(Result)
PromptResult(Success=True, Value={'x1': 12.0, 'y1': 2.0, 'x2': 5.0, 'y2': 2.0, 'label': 'linea prueba'},
Cancelled=False, Error="")
>>>
```

4. Probar PromptedCommandExecutor

Esto prueba ejecución final con parámetros:

```
from InputPrompts.PromptedCommandExecutor import PromptedCommandExecutor
state = {}
```

```
class Entry:
    InternalKey = "create_line"
```

```
    def IsCallable(self):
        return True
```

```
def CreateLine(x1: float, y1: float, x2: float, y2: float):
    state["line"] = (x1, y1, x2, y2)
```

```
Entry.Target = CreateLine
```

```
Executor = PromptedCommandExecutor(DelayMs=0)
Executor.ExecuteEntry(
    Entry(),
    SimulatedFinalTexts=[
        "uno dos enviar",
        "dos enviar",
        "cinco enviar",
        "dos enviar",
    ],
    )
```

)

```
print(state)
print(Executor.LastResult)
```

Esperado:

```
{'line': (12.0, 2.0, 5.0, 2.0)}
PromptResult(Success=True, Value={'x1': 12.0, 'y1': 2.0, 'x2': 5.0, 'y2': 2.0},
Cancelled=False, Error="")
```

```
>>> import sys
>>> sys.path.insert(0, r"D:\Facultad\PET DAV\Fork\DAV\luigiIntegracionV1\GUIFreeCad")
>>> from InputPrompts.PromptedCommandExecutor import PromptedCommandExecutor
>>> state = {}
>>> class Entry:
...     InternalKey = "create_line"
...
...     def IsCallable(self):
...         return True
...
>>> def CreateLine(x1: float, y1: float, x2: float, y2: float):
...     state["line"] = (x1, y1, x2, y2)
...
>>> Entry.Target = CreateLine
>>> Executor = PromptedCommandExecutor(DelayMs=0)
>>> Executor.ExecuteEntry(
...     Entry(),
...     SimulatedFinalTexts=[
...         "uno dos enviar",
...         "dos enviar",
...         "cinco enviar",
...         "dos enviar",
...     ],
... )
>>> print(state)
{'line': (12.0, 2.0, 5.0, 2.0)}
>>> print(Executor.LastResult)
PromptResult(Success=True, Value={'x1': 12.0, 'y1': 2.0, 'x2': 5.0, 'y2': 2.0}, Cancelled=False, Error="")
>>>
```

5. Probar Voz real con Prompt Activo

Primero asegurate de iniciar voz DAV desde la barra:

DAV -> Iniciar voz DAV

Despues abrir un prompt manual:

```
from InputPrompts.IntegerInputPrompt import IntegerInputPrompt
```

```
Prompt = IntegerInputPrompt("Integer", "Deci un entero")
Prompt.Show()
```

```
from InputPrompts.PromptVoiceRouter import PromptVoiceRouter
PromptVoiceRouter.SetActivePrompt(Prompt)
```

Ahora decí:

uno dos enviar

Esperado: el prompt debería aceptar 12.

Luego:

`print(Prompt.GetResult())`

`PromptVoiceRouter.ClearActivePrompt(Prompt)`

Esperado:

`PromptResult(Success=True, Value=12, Cancelled=False, Error="")`